MeshTac: Mesh-Aligned Pseudo-Labels for Label-Efficient Tactile Depth, Normal and Pose Estimation

Extended Version

Anonymous submission

***Abstract*— Vision-based tactile sensors provide detailed contact images, but learning to recover geometry from them requires registered training data. We present MeshTac, a method that uses known object meshes to label additional real tactile images from only a few labeled contacts. A teacher predicts an initial contact pose, and a residual aligner refines it by comparing the real image with mesh depth rendered at the current estimate. The aligner is trained by perturbing the poses of the same labeled frames; the aligned mesh then supplies depth, normal and pose targets for a student. On 12 ridge-pattern objects, using 10–25 labeled frames per object reduces contact-region depth error by 13–20% on sensor A and 33–36% on GelSlim 4.0 relative to real-only training. Each sensor is trained separately under the same label budget. Ablations show that alignment improves pseudo-label quality beyond teacher predictions or an image-error pose search. Synthetic images are unnecessary for the within-session depth gains, but improve performance in a second capture session. The same aligner also refines test-time poses, reaching 0.4–1.2° mean rotation error within the original session at $K \geq 25$. These results support mesh alignment as a practical source of supervision for tactile perception of known objects.**

I. Introduction

Vision-based tactile sensors image a soft elastomer as it deforms against an object. The resulting images contain fine contact features that are useful for robotic manipulation, including where contact occurs and how the object is oriented. Sensors such as GelSight, GelSlim and DIGIT [1]–[3] share this principle, but differ in illumination, elastomer properties and camera response. Learning to recover contact geometry therefore requires data that captures the appearance of the target sensor. Obtaining the corresponding labels is more demanding: in a mesh-based dataset, each image must be registered to the object pose.

Simulation offers one way to reduce this labeling effort. Tactile simulators generate images and geometric labels from known object meshes [4]–[6]. Their utility depends on how well the synthetic contact appearance matches the real sensor. A learned appearance model can narrow this gap, but fitting it also consumes labeled real contacts. Evaluating label efficiency therefore requires counting the labels used throughout the pipeline, including renderer training.

We study a complementary use of the mesh: assigning geometric labels directly to unlabeled real images. These images already contain the sensor's contact appearance and require no pose registration during capture. Given the object identity and mesh, the missing quantity is the contact state: in-plane rotation, translation and indentation. A model trained on a small labeled set provides an initial estimate, but its pose and dense predictions are not accurate enough to serve as reliable supervision on their own.

MeshTac refines this estimate with a learned residual aligner (Fig. 1). The aligner takes the real image and the mesh depth at a candidate state, then predicts a correction. Training examples are obtained by perturbing the known poses of the labeled frames. After refinement, mesh depth, normals and pose become targets for training a student on the additional real images. The contribution is this use of geometric supervision; teacher and student share a standard tactile perception architecture.

We evaluate MeshTac against real-only training, synthetic-data augmentation and teacher-based self-training with the same labeled frames. With 10–25 labels per object, it reduces contact MAE by 13–20% on sensor A and 33–36% on sensor B. The main contributions are:

- A mesh-aligned pseudo-labeling procedure that learns pose refinement from the available labeled contacts and supplies depth, normal and pose supervision for unlabeled real frames.
- An evaluation on two sensors, trained separately, that measures label efficiency and separates the effects of alignment, synthetic data and additional real labels. A second capture session tests sensitivity to changes in appearance and object placement.
- A test-time use of the same aligner that reduces rotation error without retraining, including on held-out objects with known meshes.

II. Related Work

Tactile simulation and transfer. TACTO [4] renders optical tactile images, Taxim [5] models optical response and marker displacement from calibration examples, and Tacchi [6] simulates elastomer deformation. Physics-based [7], differentiable [8] and large-scale simulators [9] offer different tradeoffs between contact modeling and computational cost. Learned image translation provides another route between simulated and real tactile observations [10]–[12]. Unpaired translation [13]–[15] and domain randomization [16] reduce dependence on paired data. Our experiments compare a physical renderer and a learned depth-to-image model under an explicit label budget. Sensor-invariant representations address variation at the feature level [17]–[19]; we use a frozen SITR encoder [17] for all methods.

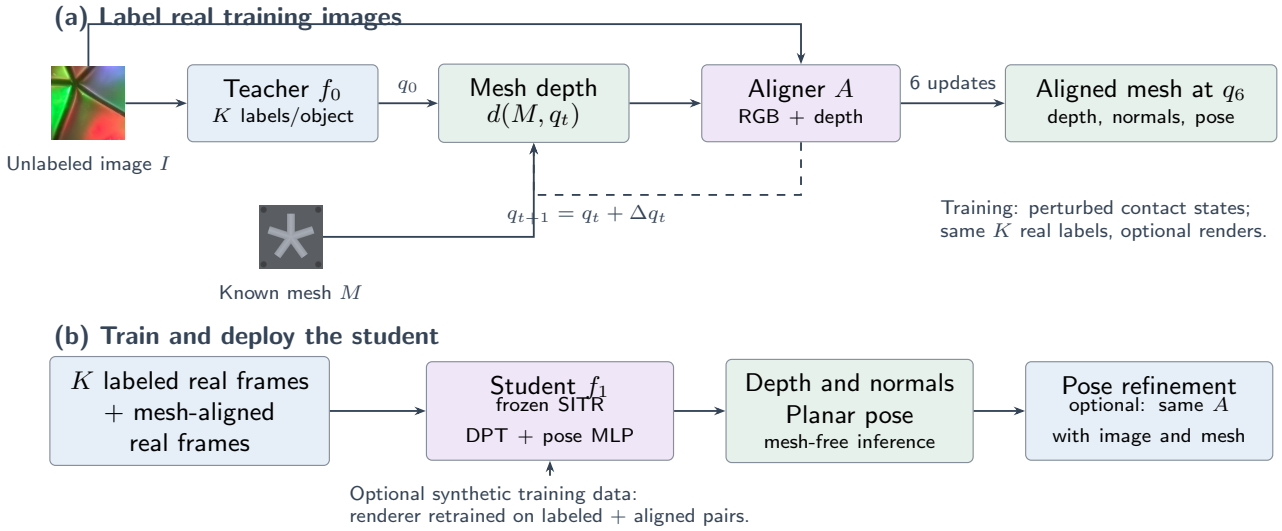


Fig. 1: **Overview of MeshTac.** A teacher initializes the contact state of an unlabeled frame. The aligner compares the frame with mesh depth and updates the state six times; the aligned mesh supplies depth, normal and planar-pose labels. Aligner training uses the same K real labels per object; synthetic augmentation is optional. The student learns from labeled and aligned real frames, optionally with synthetic images. Its forward pass requires no mesh; optional pose refinement uses the image and known mesh.

Pseudo-labeling and pose refinement. Self-training uses model predictions as supervision for unlabeled data [20], [21]. Its effectiveness depends on the quality of those predictions. In visual pose estimation, DeepIM [22] learns iterative corrections by comparing observed and rendered images, while Self6D [23] uses unlabeled real images for self-supervised pose learning. MeshTac adopts learned residual refinement for tactile contact states. It compares tactile RGB with mesh depth, learns corrections from perturbed labeled contacts, and uses the refined mesh to supervise dense tactile predictions. This labeling step requires neither a photorealistic renderer nor differentiation through rendering.

III. Setup: Sensors, Renderers and Data-Access Protocol

A. Problem Statement

A tactile renderer maps a mesh M and contact state q (planar pose and indentation) to an image and dense labels, $(\hat{I}, D, N) = \mathcal{R}_\theta(M, q)$. The geometry stage $(M, q) \mapsto (D, N)$ is shared by every renderer we study; renderers differ only in the appearance operator $\mathcal{P}_\theta : (D, N) \mapsto \hat{I}$ and in the real data $\mathcal{D}^r_{\text{cal}}$ used to fit θ. A downstream model f_ϕ predicts depth, normals and planar pose from a real image. We measure a renderer by the real test error of f_ϕ trained on its synthetic data plus a real subset $\mathcal{D}^r_{\text{task}}$, and we control the total real budget $\mathcal{D}^r_{\text{cal}} \cup \mathcal{D}^r_{\text{task}}$.

B. Sensors and Data

Sensor A. sensor A is a GelSlim-type fingertip sensor with a 17.5 mm square gel view imaged under six colored LED emitters. Images are center-cropped to 0.816 of the view and resized to 224×224.

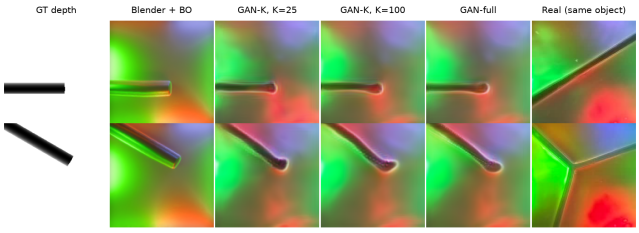


Fig. 2: **Two renderers, one contact.** Each row is one simulated contact (GT depth, left) rendered by the physically calibrated Blender renderer, by the learned renderer trained on $K = 25$ or $K = 100$ real pairs per object, and by the learned renderer trained on all real pairs (GAN-full). Right: a real frame of the same object. The physical renderer reproduces the illumination field but not the dark contact shading of the real gel; the learned renderers reproduce the shading and inherit the illumination of their training frames.

Real data (session 1). A KUKA LBR Med arm presses the sensor onto 12 ridge-pattern objects of known geometry, recording the image and end-effector pose at every contact; ground-truth (GT) depth and normals are the mesh projected at the recorded pose (geometric targets, not membrane measurements). After removing frames whose image contact region disagrees with the GT, 4,669 frames remain (222–568 per object), one session per object, every tenth frame held out as validation (472 frames).

Real data (session 2). The same objects were re-placed and re-captured on another day (48 runs). GT poses were re-estimated per frame by maximizing the IoU between the image ridge mask and the projected mesh contact and

manually reviewed; the curated test set has 854 frames of 11 objects (Sec. V-C). Nothing from this session is used for training or model selection.

Sensor B. GelSlim 4.0 [24] is the previous generation of the same fingertip family, with a marker array, a different illumination layout and a 12 mm field of view. Its public depth dataset [24] has the same 12 objects pressed by a parallel gripper at 1 mm indentation with mesh-derived depth and in-hand pose labels (135 test frames per object). We use the left finger, the central 12×12 mm, and report rotation after removing a per-object constant offset because its pose origin differs from ours. Its depth labels are clipped at the nominal indentation and smoothed, so they are not exact mesh projections. For the MeshTac experiments of Sec. V-E we therefore re-registered them: for every frame the planar mesh pose that best explains the public label was found by a global search (0.018 mm mean residual) and canonicalized against the rotational symmetry of the object, and the mesh depth at that pose replaces the label for training and testing (*mesh-registered* labels). The renderer baselines of Sec. V-E are reported with the original labels.

Contact sampler. For each object we sample contact cells over the mesh at 12 base rotations (30° apart), an indentation of 0.8–1.2 mm and in-plane jitter, and export D from the object z-buffer and N by finite differences. Simulated poses are restricted to each object's real rotation and translation ranges. The same $5{,}345$ simulated contacts feed every renderer.

C. Physically Calibrated Renderer

The physical renderer is implemented in Blender/Cycles: a gel surface mesh is shrink-wrapped onto the object and smoothed (35 corrective iterations), shaded with a glossy/transparent material and a height-driven contact-darkening term, lit by six area emitters, and post-processed (saturation, brightness, contrast, haze, a low-frequency random gel warp). Emitter and post-processing parameters (16 scalars) are fitted by tree-based Bayesian optimization of an L_2 plus histogram distance between the real and rendered no-contact image (500 evaluations); gel and contact parameters were hand-tuned on a few contact frames. Thus, one no-contact image is the automatic calibration target, but additional contact images informed manual tuning. Their use must be distinguished from the counted paired training labels.

Domain-randomization baseline. To test whether careful calibration is necessary, we also render with randomized sensor parameters: random HSV colors and strengths for each LED emitter, random gel optics, per session, with the geometry stage (scale, field of view, smoothness) fixed. This variant uses no image to fit its randomized appearance parameters; it retains the shared geometry configuration.

D. Learned Renderer

The learned renderer replaces $\mathcal{P}_\theta$ by a conditional GAN [25]: a five-level U-Net that takes the depth map and two coordinate channels and outputs an RGB image, a 70×70 spectral-normalized PatchGAN discriminator, and an adversarial $+\,100\,(\mathrm{L1}+\mathrm{gradient\ L1}+\mathrm{low\text{-}frequency\ L1})$ loss. Simulated depth is jittered by $\pm 15\%$ at generation time.

Real budget. GAN-K is trained on K real (depth, image) pairs per object, an evenly spaced subset of each object's training frames that is *the same* K frames the downstream model receives; GAN-LOO on all training frames of the 9 non-held-out objects; GAN-full on all $4{,}197$ pairs. Small budgets train for at least 30k generator updates.

Variants. *Background-conditioned*: the session's no-contact image is appended as three input channels, so a new session costs one background frame, as for the physical renderer. *Physics-conditioned*: the physical renderer's image of the same contact is appended, optionally predicting only a residual on top of it, optionally with the background. *Difference image*: the target is the image minus the session background. *Blender-pretrained*: pretraining on 30k (depth, Blender image) pairs, then fine-tuning on the real pairs. *Multi-sensor*: the background-conditioned GAN is trained on 100 simulated sensor designs (random LED layouts and gel materials) plus optionally the target sensor's real pairs; this tests whether sensor diversity can replace sensor-specific real data. *Rotation coverage* changes the geometry stage instead: we vary how many of an object's real rotations appear in simulation (1, 2 or all 12) and, for the learned renderer, rotate the depth map before rendering so that every rotation is covered.

E. Zero-Label Alternative: Contact-Preserving Unpaired Refiner

MeshTac (Sec. IV) needs K labeled pairs to start. A renderer that needs *no labeled pair at all* is conceivable: geometry transfers through the contact silhouette, which the physical renderer already provides, and what is missing is the shading inside the contact, which might be learned from unlabeled real frames alone, as in unpaired image translation [11], [13], [15]. We test this alternative. The refiner $\mathcal{F}$ takes the physical render $\hat{I}_{\mathrm{phys}}$, the session background B and the contact mask m (simulated depth > 0, dilated by 4 px and feathered) and outputs

$$\hat{I} = m \odot \mathcal{F}(\hat{I}_{\mathrm{phys}}, B, m) + (1 - m) \odot B, \qquad (1)$$

so the background is the real one and the contact region is the only learned part; depth, normal and pose labels are those of the physical render by construction. $\mathcal{F}$ is the same U-Net as the learned renderer with a 7-channel input. It is trained against a 70×70 PatchGAN on *unlabeled* real frames (image only; no depth, no pose) with an adversarial loss and a patchwise contrastive loss (PatchNCE [15]) between the encoder features of input and output, sampled preferentially inside m, which keeps the ridge structure of the render while its shading is translated; 10k generator updates, batch 8, Adam (2×10^{-4}, $\beta_1 = 0.5$). Its real budget is the set of unlabeled frames plus one background frame per session, and the same leakage rule applies: in leave-object-out the held-out objects' frames are excluded, in K-shot the same K frames as the downstream model are used without their labels. Applied to the domain-randomized render, the pipeline uses no calibration and no label at all.

F. Downstream Tactile Model

The downstream model is identical for every renderer: a frozen sensor-invariant encoder (SITR-B/18 [17]) provides patch tokens at four depths; a DPT decoder [26] predicts depth and normals at 224×224; a pose head regresses $(\cos\theta, \sin\theta, t_x, t_y)$ from the class token and mean patch token. For the classification study, we add a 12-class head (LayerNorm on concatenated class and mean patch tokens, two-layer MLP with GELU) and train with cross-entropy alongside the other losses. Only decoder, pose head and an object embedding are trained (12M parameters). Losses are MSE on depth and normals, $1 - \cos(\theta - \hat{\theta})$ on rotation and L1 on translation, balanced by grouped uncertainty weighting [27]. Training co-mixes simulated and real frames; simulated frames receive photometric augmentation and $\pm 15°$ gel-spin rotation with matched labels, real frames none. AdamW, learning rate 2×10^{-4}, batch 32, cosine schedule, 150 epochs for the main sensor-A runs, 50 for sensor B and seed/ablation studies. Depth and normal metrics use the checkpoint with the lowest validation depth MSE, sensor-A pose metrics in the main within-session and pose-refinement tables use the one with the lowest validation angular error. The legacy cross-session table reports pose from the depth checkpoint. Checkpoint selection uses validation labels in addition to the K training labels.

G. Data-Access Protocol

Paired renderers and the aligner exclude validation and test images from training. The K-shot protocol additionally limits paired training labels to the same K frames per object. Validation labels remain available for model selection; the reported budget counts training labels. Physical-renderer manual tuning is described separately above.

Leave-object-out (LOO). Three objects (a two-line, a rod and a cross pattern; 1,244 frames) are removed from all real training, excluded from paired-renderer and aligner training, and evaluated in full. Simulated contacts of the three meshes are available to every renderer ("new object with a CAD model but no real touches"). A *geometry control* additionally removes the three meshes from simulation.

K-shot. Every object contributes $K \in \{0, 10, 25, 50, 100\}$ evenly spaced real training frames. The learned renderer is trained on exactly those frames; the downstream model co-trains on them plus simulated data, or on them alone with the same number of updates. Evaluation is on the real validation split. The aligner of MeshTac receives the same K frames, and the remaining training frames may be used *without* their labels.

Cross-session and cross-sensor. Session-2 and sensor-B test frames are never used to train or select anything in the sensor-A study. In the sensor-B study the K-shot protocol is repeated with sensor-B real pairs and the sensor-B test split.

Leaky reference. GAN-full is trained on all 4,197 sensor-A training pairs. In LOO, these include real pairs from held-out objects; in K-shot, they include training labels withheld from budget-matched methods, but no validation frames. We retain the label "leaky renderer" for this non-comparable reference, shown in italics and excluded from best-result highlighting.

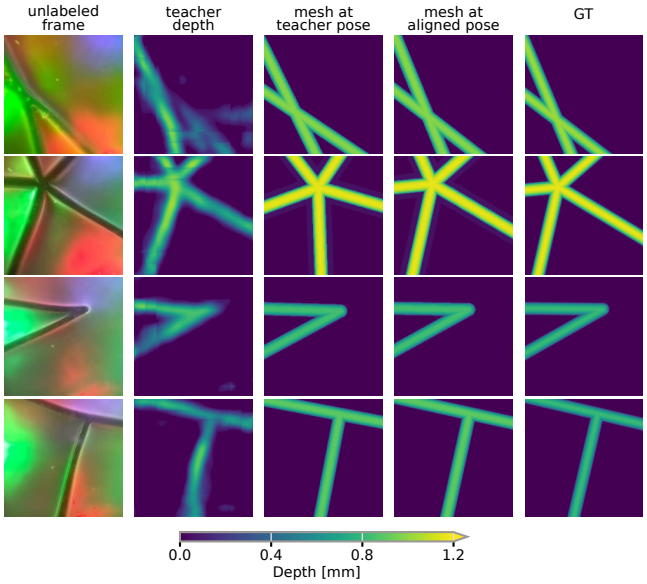


Fig. 3: **Pseudo-label construction.** Sensor A, $K = 25$. Columns compare teacher depth and mesh projections before and after alignment with GT. Each row is a median-IoU example from a subsampled candidate set for one object. All depth maps use the same scale (mm).

Metrics. Depth MSE over all pixels (mm^2), the metric of prior work on this sensor; and, because $\approx 90\%$ of pixels are background, the mean absolute error inside the GT contact region (contact MAE, mm) and the contact IoU (predicted depth > 0.05 mm vs. GT > 0), which also penalizes false contact. Normals: mean angular error inside the contact region (deg). Pose: mean angular error (deg) and normalized translation L1.

IV. MeshTac: Mesh-Aligned Pseudo-Labeling

Geometric labels. Let $d(M, q)$ denote the depth of object mesh M at contact state $q = (\theta, x, y, z)$, where z is indentation. For a known object, estimating these four variables determines a mesh-derived depth map, its normals and the planar pose. MeshTac uses this constraint to label real frames whose capture poses are unavailable. Mesh geometry remains fixed; only the contact state is refined.

Learning the aligner. A ResNet-18, initialized from ImageNet, takes tactile RGB and candidate mesh depth as four input channels. It predicts a residual A in contact-state units:

$$A\big(I, d(M, q^* + \delta)\big) \approx -\delta, \tag{2}$$

where q^* is the labeled state and δ a sampled perturbation. The ranges are $\pm 8°$, ± 1.5 mm in-plane and ± 0.15 mm in indentation; half the samples use $0.3\times$ these ranges. Residuals are normalized by these ranges for a smooth-L1 training loss (8,000 steps, batch 64, AdamW, learning rate 3×10^{-4}). The main sensor-A aligner mixes 24 real and 40 synthetic

examples per batch, with synthetic images generated by the K-pair renderer. The renderer-free and sensor-B aligners use real examples only. Both reuse the same K real labels.

Labeling real images. For each unlabeled training image, teacher f_0 initializes planar pose and sets indentation to its maximum predicted depth. On sensor A the teacher uses the K real frames and learned-renderer data; the renderer-free variant and sensor B use a real-only teacher. Given the known mesh, we perform six updates,

$$q_{t+1} = q_t + A\big(I, d(M, q_t)\big), \tag{3}$$

clipping indentation to $[0.3, 1.6]$ mm. The final mesh depth, normals and planar pose label the image. We retain all training frames in the evaluated configuration (Sec. V-B).

Training the student. Student f_1 has the teacher's architecture and learns from the K labeled frames and the mesh-aligned frames. The sensor-A configuration also uses synthetic images from a renderer retrained on labeled and aligned real pairs. We evaluate a version with no renderer at any stage to isolate alignment. No validation or test frame contributes a pseudo-label to training.

Optional test-time refinement. The student predicts from an image without mesh rendering. When the object identity and mesh are available, the same fixed aligner can refine its predicted pose using the test image. This is an additional inference step, evaluated separately in Sec. V-D.

V. Experiments

Sensor-A main runs use 150 epochs; seed checks, ablations and sensor-B runs use 50 (other exceptions are noted). Metrics are evaluated on real images; depth MSE in mm^2, contact MAE in mm, normals and rotation in degrees. Sections V-A–V-F evaluate MeshTac; Sections V-G–V-I report the leakage-controlled renderer study that provides its baselines.

A. Label Efficiency

Tables I and II, and Fig. 4 compare, at equal label budget, real-only training, co-training with either renderer, self-training on the teacher's own predictions (Sec. V-B) and MeshTac. Each MeshTac run uses the 2,997–4,077 training frames outside the K budget without their labels.

Simulation alone. Neither paired-renderer baseline improves both contact MAE and IoU for $K \leq 25$; the learned renderer helps from $K = 50$ (depth MSE -10%, -16% at $K = 100$), the physical renderer by at most 5%. Both lower rotation error at $K = 10$ ($9.6^\circ \rightarrow$5–6°) and raise it at $K = 100$.

MeshTac improves on real-only training at every budget: depth MSE by 13%, 19%, 36% and 37% and contact MAE by 13%, 20%, 27% and 28% at $K = 10$, 25, 50 and 100, with IoU $+0.06$ to $+0.08$ and normal error -2.6° to -5.1°. Over three seeds (50 epochs) the depth MSE is 0.0558 ± 0.0003, 0.0447 ± 0.0009, 0.0313 ± 0.0007 and 0.0229 ± 0.0003 against 0.0635 ± 0.0007, 0.0551 ± 0.0010, 0.0480 ± 0.0006 and 0.0366 ± 0.0008 for real-only training. Three comparisons put the gain in perspective. MeshTac with 50 labels per object (0.0307) is ahead of real-only training with 100 (0.0369) and on par with the learned renderer at 100 (0.0311). At $K = 100$ it is *better* than the leaky renderer (depth MSE 0.0232 vs. 0.0274, contact MAE 0.168 vs. 0.205), and at $K = 25$ it has lower contact MAE (0.249 vs. 0.262) and slightly higher IoU, despite 8% higher depth MSE. Its rotation error is the lowest leakage-free value for $K \leq 50$ (4.0°, 3.7°, 3.7°); at $K = 100$ real-only training is ahead (3.2° vs. 3.7°) until the aligner is also used at test time (Sec. V-D). The depth gains persist without a renderer: with a real-only teacher, an aligner trained on the K frames alone and no synthetic frame at all ("no renderer"), depth MSE is 0.0521, 0.0433, 0.0291 and 0.0227 and rotation error 2.5–4.7°, equal to or better than with the renderer; the renderer provides an additional benefit across sessions (Sec. V-C).

Seeds. Table III repeats the main comparison with three seeds: the standard deviations are an order of magnitude below the differences between methods.

B. Why Alignment: Ablation

Table IV compares pseudo-labels and student configurations, including whether the renderer is retrained or omitted.

Self-training. With the teacher's own depth as label (stitched from five windows to cover the full frame, pose loss on the labeled frames only), depth MSE improves by 8% but contact MAE and IoU do not: the pseudo labels inherit the teacher's localization error (label IoU 0.56), indicating that a lower all-pixel error need not imply improved localization. Adding the teacher's pose as a label improves rotation accuracy with little change in dense metrics, and a second round of self-training changes depth MSE by at most 4% in either direction.

L1 render-and-compare. Replacing the aligner by a search—fit the mesh to the teacher depth on a pose grid, then pick, among 567 neighbouring poses, the one whose GAN-K render is closest to the frame in image L1—helps less (label IoU 0.63 with the half-frame gate, pose error 2.5°) and requires gating: on the half of the frames with the larger render error the mesh label is no better than the teacher's. The few-shot renderer is a poor comparator; at $K = 25$ its render at the true pose is farther from the frame than its render at the selected pose on 93% of the frames.

Learned aligner. The aligner lowers the teacher's pose error from 5.3° to 1.1° on the 3,897 unlabeled frames and yields labels with IoU 0.71 and contact MAE 0.24 mm (teacher: 0.55, 0.32). Label quality grows with the budget (IoU 0.65, 0.71, 0.76, 0.81 and pose error 1.6°, 1.1°, 1.1°, 0.7° for $K = 10$–100). Relative to teacher-depth self-training, the student lowers contact error ($0.310 \rightarrow 0.246$) and raises IoU ($0.624 \rightarrow 0.729$). Further findings: (i) the aligner can be trained on real examples alone (label IoU 0.72), while the main mixed real/synthetic configuration gives 0.71; synthetic-only training is less accurate (0.56); (ii) no gate is needed, since on the frames a render-error gate would reject its labels still beat the teacher's (IoU 0.66 vs. 0.55); (iii) label IoU changes little across the tested settings (half or double the perturbation range: 0.72, 0.73; 2k or 20k steps: 0.73, 0.72; ResNet-34 or -50 backbone: 0.73, 0.71), while dropping the ImageNet initialization costs accuracy (0.67); (iv) the student is unchanged whether its synthetic frames come from

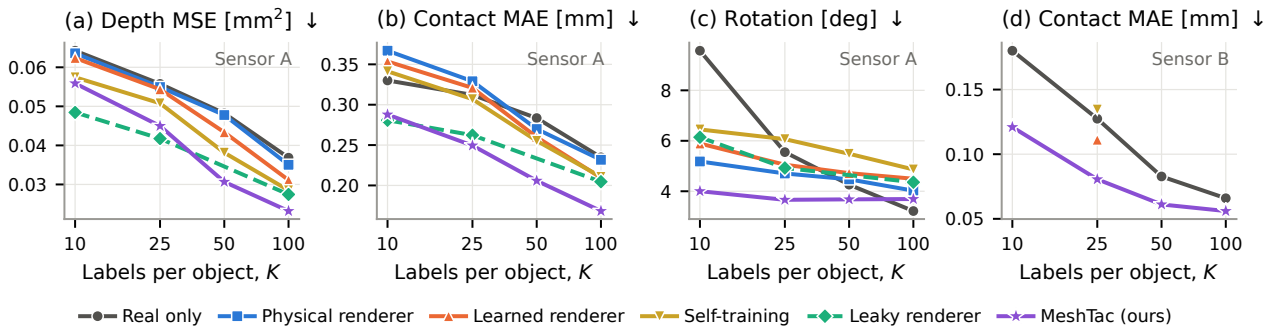


Fig. 4: **Label efficiency.** (a–c) Sensor-A validation results; (d) sensor-B test results with renderer-free MeshTac. Lower is better. Lines connect evaluated budgets. Sensor A uses 150 epochs except the $K = 10$ leaky reference (50); sensor B uses 50.

TABLE I: **K-shot with a shared real budget.** Every object contributes K real frames; GAN-K is trained on the same frames. MeshTac and self-trained rows additionally use the unlabeled training frames (Sec. V-B); refiner training uses no paired label. Real validation split (472 frames). Leaky rows use the renderer trained on all real pairs (the $K = 10$ leaky row is a 50-epoch run).

K	Method	Depth MSE↓	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓
0	Blender only	**0.0793**	**0.408**	**0.572**	**48.6**	**8.4**
	GAN-full (leaky) only	*0.0459*	*0.274*	*0.705*	*44.6*	*7.0*
10	Real only	0.0642	0.330	0.625	29.8	9.6
	Blender	0.0636	0.367	0.602	30.8	5.2
	GAN-10	0.0624	0.354	0.597	30.6	5.9
	Teacher-label self-training	0.0574	0.341	0.592	28.3	6.5
	MeshTac, L1 search	0.0545	0.320	0.639	27.3	4.6
	MeshTac (with renderer)	0.0559	**0.288**	**0.682**	27.2	4.0
	no synthetic frames	0.0571	0.293	0.678	27.3	**3.2**
	MeshTac (no renderer)	**0.0521**	0.293	0.660	**26.5**	4.7
	GAN-full (leaky)	*0.0484*	*0.281*	*0.693*	*28.5*	*6.1*
25	Real only	0.0558	0.312	0.664	28.2	5.6
	Blender	0.0549	0.329	0.656	27.5	4.7
	GAN-25	0.0544	0.321	0.626	28.3	5.1
	Blender ⊕ GAN-25	0.0554	0.324	0.667	27.4	4.8
	Refiner-25 (0 labels)	0.0551	0.297	0.672	27.1	5.5
	Refiner-U (0 labels, all unlabeled)	0.0566	0.309	0.666	27.8	5.6
	Self-trained GAN-25 (renderer only)	0.0538	0.326	0.641	28.0	4.8
	Teacher-label self-training, model only	0.0517	0.300	0.611	26.5	8.2
	center window only (no stitching)	0.0518	0.328	0.643	26.7	8.4
	Teacher-label self-training	0.0508	0.307	0.628	26.5	6.1
	round 2	0.0500	0.310	0.642	26.0	6.2
	MeshTac, L1 search	0.0482	0.291	0.655	25.5	4.5
	MeshTac (with renderer)	0.0450	0.249	**0.727**	24.0	3.7
	no synthetic frames	0.0456	0.259	0.715	24.7	**2.7**
	MeshTac (no renderer)	**0.0433**	**0.245**	0.726	**23.3**	2.8
	GAN-full (leaky)	*0.0417*	*0.262*	*0.724*	*25.5*	*4.9*

the renderer retrained on aligned pairs or on teacher-labeled pairs (depth MSE 0.0447 in both cases), and within the session it does not need them at all (last row: 0.0433 ± 0.0004 over three seeds with a real-only teacher and no synthetic frame, against 0.0447 ± 0.0009 with the renderer); (v) a quarter of the unlabeled frames already gives most of the gain (depth MSE 0.0453, 0.0452 and 0.0447 with 25%, 50% and 100% of U), while IoU keeps growing with their number (0.710, 0.725, 0.729); (vi) a second round, with the student as the new teacher, does not improve further: although the student proposes poses 1.5° closer than the original teacher, the aligned labels (IoU 0.65, 0.71 at $K = 10$, 25) and the retrained student (depth MSE 0.0570, 0.0445; IoU 0.674, 0.717) are within seed variation of the first round. An unpaired sim-to-real refiner of the physical render, which uses the unlabeled frames without any pose, does not improve on its input (contact MAE 0.349 vs. 0.346 on unseen objects).

Self-trained renderer. Following self-training [20], [21], the

TABLE II: **Larger training-label budgets.** Continuation of Table I, $K = 50$ and 100. All data-access and metric conventions are unchanged.

K	Method	Depth MSE↓	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓
50	Real only	0.0482	0.283	0.690	25.5	4.3
	Blender	0.0478	0.270	0.701	25.2	4.5
	GAN-50	0.0434	0.260	0.716	24.3	4.7
	Teacher-label self-training	0.0381	0.255	0.706	22.7	5.5
	MeshTac, L1 search	0.0384	0.247	0.714	22.7	4.1
	MeshTac (with renderer)	0.0307	0.206	**0.769**	20.4	3.7
	no synthetic frames	0.0332	0.215	0.764	21.1	3.0
	MeshTac (no renderer)	**0.0291**	**0.197**	0.761	**19.8**	**2.7**
100	Real only	0.0369	0.235	0.744	22.1	3.2
	Blender	0.0350	0.232	0.727	22.0	4.0
	GAN-100	0.0311	0.210	0.773	20.8	4.5
	Blender ⊕ GAN-100	0.0329	0.213	0.760	20.5	4.2
	Refiner-100 (0 labels)	0.0340	0.224	0.761	21.1	4.9
	Refiner-U (0 labels, all unlabeled)	0.0329	0.228	0.747	21.1	4.9
	Self-trained GAN-100 (renderer only)	0.0308	0.207	0.774	20.6	4.4
	Teacher-label self-training, model only	0.0290	0.221	0.744	19.8	4.5
	Teacher-label self-training	0.0285	0.211	0.750	19.7	4.9
	round 2	0.0295	0.217	0.736	20.2	4.9
	MeshTac, L1 search	0.0276	0.203	0.771	19.4	4.1
	MeshTac (with renderer)	0.0232	0.168	0.803	17.1	3.7
	no synthetic frames	0.0237	0.168	0.809	17.1	2.7
	MeshTac (no renderer)	**0.0227**	**0.166**	**0.818**	**16.9**	**2.5**
	GAN-full (leaky)	*0.0274*	*0.205*	*0.777*	*19.9*	*4.4*

TABLE III: **Seeds.** Mean ± standard deviation over three seeds (50 epochs) on the real validation split. "MeshTac": real-only teacher, no synthetic frame; "with renderer": the configuration of Table I.

K	Method	Depth MSE	Cont. MAE	IoU	Rot.
25	Real only	0.0551 ± 0.0010	0.314 ± 0.010	0.665 ± 0.004	5.5 ± 0.0
25	GAN-25	0.0536 ± 0.0013	0.314 ± 0.008	0.655 ± 0.032	6.0 ± 0.1
25	Teacher-label self-training	0.0507 ± 0.0001	0.310 ± 0.004	0.624 ± 0.002	7.2 ± 0.1
25	MeshTac, L1 search	0.0482 ± 0.0004	0.289 ± 0.004	0.648 ± 0.010	5.5 ± 0.1
25	MeshTac (with renderer)	0.0447 ± 0.0009	0.246 ± 0.002	0.729 ± 0.002	4.9 ± 0.1
25	MeshTac	0.0433 ± 0.0004	0.247 ± 0.000	0.722 ± 0.000	4.0 ± 0.0
100	Real only	0.0366 ± 0.0008	0.234 ± 0.006	0.750 ± 0.004	3.9 ± 0.0
100	GAN-100	0.0299 ± 0.0004	0.201 ± 0.002	0.779 ± 0.001	5.5 ± 0.0
100	Teacher-label self-training	0.0280 ± 0.0001	0.209 ± 0.002	0.743 ± 0.007	6.1 ± 0.0
100	MeshTac, L1 search	0.0275 ± 0.0002	0.201 ± 0.003	0.767 ± 0.005	5.0 ± 0.1
100	MeshTac (with renderer)	0.0229 ± 0.0003	0.164 ± 0.000	0.818 ± 0.000	4.8 ± 0.0
100	MeshTac	0.0222 ± 0.0004	0.163 ± 0.002	0.821 ± 0.002	3.8 ± 0.0

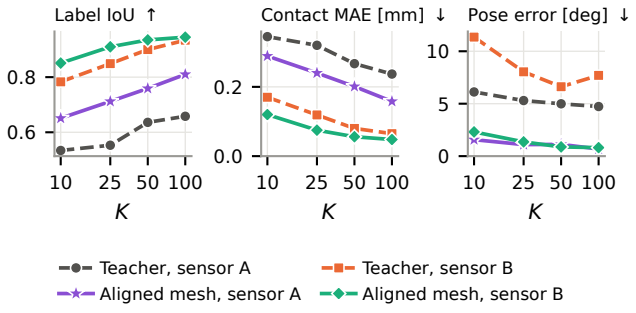


Fig. 5: **Pseudo-label quality.** Teacher predictions and aligned mesh labels, evaluated against withheld training GT on each sensor. The withheld GT is not used for learning.

K-shot downstream model pseudo-labels U (depth predicted on five windows and stitched back to the full frame; frames kept when the predicted contact overlaps the image's black-hat ridge mask, IoU ≥ 0.2: 3,260 frames at $K = 25$, 2,677 at $K = 100$), and GAN-K is retrained on the K true plus the pseudo pairs. The pseudo labels are as accurate as the labeler (0.32 and 0.24 mm contact MAE against the withheld GT labels), and the renderer's fidelity improves by 5% at both budgets ($0.108 \rightarrow 0.102$, $0.077 \rightarrow 0.073$), which the proxy translates into a 1% downstream gain: depth MSE $0.0544 \rightarrow 0.0538$ at $K = 25$ and $0.0311 \rightarrow 0.0308$ at $K = 100$ (contact MAE 0.207, lower than the paired-renderer baseline at $K = 100$). With teacher labels the renderer thus gains little from more real images. With the aligned pairs it gains more: contact-region error falls by 13–19% at every

TABLE IV: **Alignment ablation** ($K = 25$, 50 epochs; first, fifth and last row are three-seed means). Left: quality of the pseudo labels of the unlabeled frames against their withheld GT labels (evaluation only). Right: real validation error of the model trained on them. Depth label: teacher prediction or mesh depth at the fitted pose ($^{1/2}$: mesh on the half of each object's frames with the lowest render error, teacher on the rest). Pose label: none (pose loss on the K labeled frames only), teacher pose, pose found by L1 render-and-compare search with GAN-K ($^{1/2}$: on the confident half only), or the learned aligner. ‡: synthetic data from the teacher-pair renderer; †: MeshTac (renderer retrained on the aligned pairs).

Labels of U		Pseudo-label quality			Downstream			
		IoU	Cont.	Rot.	Depth	Cont.	IoU	Rot.
Depth	Pose	↑	MAE↓	(deg)↓	MSE↓	MAE↓	↑	(deg)↓
teacher	–	0.56	0.32	–	0.0507	0.310	0.624	7.2
teacher	teacher	0.56	0.32	4.9	0.0508	0.310	0.622	5.9
mesh$^{1/2}$	L1$^{1/2}$	0.63	0.30	1.4	0.0485	0.286	0.639	6.3
mesh	L1	0.64	0.30	2.5	0.0489	0.292	0.657	5.4
mesh$^{1/2}$	L1	0.63	0.30	2.5	0.0482	0.289	0.648	5.5
mesh	aligner‡	0.71	0.24	1.1	0.0447	0.248	0.730	5.0
mesh	aligner†	0.71	0.24	1.1	0.0447	0.246	0.729	4.9

budget ($0.121 \to 0.105$ at $K = 10$, $0.108 \to 0.090$ at 25, $0.095 \to 0.077$ at 50, $0.077 \to 0.065$ at 100; leaky 0.057), the best strict renderers of the study, although swapping this renderer for the teacher-pair one leaves the downstream error of the aligned model unchanged (Table IV): once the real frames carry good labels, the synthetic frames matter little.

Window stitching. Without alignment the label is the teacher depth, and labeling U on the center window alone leaves the pseudo depth empty outside the 0.816 crop (pseudo-label contact MAE 0.41 mm over the full frame instead of 0.32). The self-trained model then has the same depth MSE (0.0518 vs. 0.0517, model only, $K = 25$; Table I) but a worse contact MAE (0.328 vs. 0.300) and a 30% larger peak-depth error, and the gap persists across sessions (contact MAE 0.358 vs. 0.318): the rotation augmentation brings the frame border into the training crop, where unstitched labels are wrong.

Rounds. Without alignment, relabeling U with f_1 and repeating the renderer and model steps improves the pseudo labels at $K = 25$ (label IoU $0.56 \to 0.60$, contact MAE $0.32 \to 0.31$ mm, 3,516 frames kept instead of 3,260) but not at $K = 100$ ($0.66 \to 0.65$), and the downstream change is small in both directions: depth MSE $0.0508 \to 0.0500$ and IoU $0.628 \to 0.642$ at $K = 25$, $0.0285 \to 0.0295$ and $0.750 \to 0.736$ at $K = 100$, rotation unchanged (6.2°, 4.9°). Additional self-training rounds have limited benefit in these runs. A second MeshTac round also changes the result little (item (vi) above).

C. A Second Session

On the second session (Table V) dense error is 1.5–3× the same-session value, partly because the GT poses are re-estimated. The ordering of the methods is preserved. MeshTac has the lowest contact MAE and the highest IoU of its budget (0.310, 0.660 at $K = 25$; 0.252, 0.743 at $K = 100$), i.e. 16% and 28% less contact error than real-only training, relative to the matched real-only baselines. The nine-object baseline (0.315, 0.675) has different object coverage and is reported for context, not as a matched label-efficiency comparison. Simulation also helps across sessions (contact MAE $0.315 \to$ 0.27–0.29 and rotation $16.4° \to$ 10.5–11.5° for the nine-object model). Here the synthetic frames matter: without the renderer MeshTac has higher contact MAE (0.288 vs. 0.252) and lower IoU (0.707 vs. 0.743) at $K = 100$; it still improves on self-training in MAE (0.300) and matches its IoU, although it is as accurate within the session—the renderer's contribution is robustness to the appearance of a new session, consistent with Sec. V-I. Raw rotation error

TABLE V: **Cross-session.** Sensor-A models evaluated on the curated session-2 set (854 frames, 11 objects; poses re-estimated per frame and manually reviewed). Rot.: raw; Rot.*: after removing a per-object constant offset (the re-placement of the object). All models use the full session-1 real set unless a K is given.

Method	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓	Rot.* (deg)↓
Real only (9 objects, session 1)	0.315	0.675	25.4	16.4	6.5
Blender	0.294	0.701	24.9	11.5	5.1
Blender (domain rand.)	0.294	0.691	25.5	12.3	5.9
Blender, bg-norm.	0.291	0.698	24.9	**9.7**	**4.7**
GAN-LOO	0.272	0.723	24.1	11.1	5.2
Bg-cond. GAN-LOO	0.270	0.727	24.2	10.5	4.7
Blender ⊕ GAN-LOO	0.275	0.724	24.0	9.8	4.8
Refiner (0 labels)	0.290	0.714	24.6	12.0	5.3
GAN-full (leaky)	*0.259*	*0.748*	*22.9*	*10.1*	*4.8*
Blender only ($K = 0$)	0.425	0.561	50.7	25.2	5.6
$K = 25$ real only	0.368	0.598	30.7	20.8	5.4
$K = 25$ + Blender	0.382	0.615	30.1	23.0	5.4
$K = 25$ + GAN-25	0.342	0.589	28.7	25.7	5.3
$K = 25$ + refiner-25 (0 labels)	0.338	0.635	29.5	15.5	5.6
$K = 25$ + self-trained GAN-25	0.363	0.606	29.1	20.4	5.5
$K = 25$ + teacher-label self-training	0.335	0.598	27.5	13.1	5.5
$K = 25$ + MeshTac, L1 search	0.322	0.621	27.4	17.2	5.1
$K = 25$ + MeshTac	0.310	0.660	27.2	13.8	5.3
$K = 100$ real only	0.351	0.657	27.5	17.5	5.5
$K = 100$ + Blender	0.299	0.647	25.3	16.4	5.1
$K = 100$ + GAN-100	0.295	0.713	25.6	13.7	5.4
$K = 100$ + refiner-100 (0 labels)	0.330	0.688	26.2	16.9	6.1
$K = 100$ + self-trained GAN-100	0.310	0.715	26.1	12.1	4.9
$K = 100$ + teacher-label self-training	0.300	0.707	24.2	11.0	4.9
$K = 100$ + MeshTac, L1 search	0.293	0.718	23.5	12.5	5.0
$K = 100$ + MeshTac	**0.252**	**0.743**	**22.8**	14.4	5.2

TABLE VI: **Test-time alignment.** Rotation error (deg) of the predicted pose and after six aligner iterations against the object mesh. No retraining.

	Validation (session 1)				Session 2			
	Real only		MeshTac		Real only		MeshTac	
K	pred.	+align	pred.	+align	pred.	+align	pred.	+align
10	9.6	2.8	4.0	1.5	20.2	11.4	8.8	4.9
25	5.6	1.2	3.7	0.9	16.2	5.5	9.0	4.8
50	4.3	1.2	3.7	1.2	13.8	4.8	10.6	5.2
100	3.2	0.4	3.7	0.8	12.9	4.9	10.2	4.8

remains above $10°$ for every method because the objects were re-placed; two thirds of it is a per-object constant.

Renderer baselines across sessions. Table V evaluates the sensor-A models on the second capture. Dense error is 1.5–$3\times$ the same-session value (e.g. contact MAE 0.315 mm against 0.117 for the model trained on all session-1 training data from nine objects), partly because the GT poses are re-estimated; the informative quantity is the ordering. Simulation helps every task across sessions. With the nine-object session-1 real set, contact MAE falls from 0.315 mm (real only) to 0.29 with the physical and 0.27 with the learned renderer, IoU rises from 0.675 to 0.69–0.70 and 0.72–0.73, normal error falls from $25.4°$ to 24.0–$24.9°$ with the calibrated and learned renderers, and raw rotation error drops from $16.4°$ to 9.7–$11.5°$; the geometry control ($28.7°$) shows that the pose gain again comes from mesh coverage. The domain-randomized renderer matches the calibrated one on dense error (0.294, IoU 0.691) and is slightly behind on pose ($12.3°$), as on unseen objects: randomized appearance degrades the contact-silhouette signal. About $2/3$ of the remaining rotation error is a per-object constant (Rot.* 4.7–$6.5°$ for every model), i.e. the re-placement offset rather than a perception error. In the K-shot regime the dense gain grows with K: at $K = 25$ only the learned renderer lowers contact MAE ($0.368 \rightarrow 0.342$), at $K = 100$ both renderers do ($0.351 \rightarrow 0.295$–0.299) and the learned one also raises IoU ($0.657 \rightarrow 0.713$) and lowers rotation error ($17.5° \rightarrow 13.7°$), whereas simulation-only training does not transfer dense geometry across sessions (IoU 0.56, normal $51°$): the session-invariant part of the synthetic data is its geometry, not its appearance.

D. Test-Time Alignment

Since pose is defined with respect to a known object, the K-shot aligner can refine the pose predicted on a test frame (Table VI). On the validation split rotation error falls from 3.2–$9.6°$ to 0.4–$2.8°$ for real-only models and from 3.7–$4.0°$ to 0.8–$1.5°$ for MeshTac models; from $K = 25$ on the refined error is about $1°$ whichever model proposes the pose, and translation error falls by 25–50%. On the second session rotation error falls from 13–$20°$ to 5–$11°$ (real only) and from 9–$11°$ to about $5°$ (MeshTac) with a median of 1.5–$2.2°$: the mean is dominated by a minority of frames on which the alignment fails. The mesh at the aligned pose is also a depth estimate: on the validation split it is as accurate as the MeshTac student's output (IoU 0.81 vs. 0.79, contact MAE 0.158 vs. 0.168 mm at $K = 100$) and lifts a real-only model to the MeshTac level (IoU $0.656 \rightarrow 0.710$ at $K = 25$), so the student distills into a mesh-free network what alignment achieves with the mesh. The aligner also generalizes across objects: trained on the nine seen objects of the leave-object-out protocol and applied to the three held-out ones, whose only available data is their mesh, it lowers the rotation error of the simulation-trained models of Table IX from 4.8–$6.8°$ to 0.6–$1.0°$.

TABLE VII: **Second sensor (GelSlim 4.0) with mesh-registered labels**; $1{,}620$ test frames, 50 epochs. "no renderer": real-only teacher and no synthetic frame. Self-training uses the teacher's own depth as label.

K	Method	Depth MSE↓	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓
10	Real only	0.0314	0.180	0.780	22.5	14.0
	MeshTac (ours), no renderer	**0.0170**	**0.121**	**0.856**	**19.7**	**9.5**
25	Real only	0.0182	0.127	0.847	19.6	13.4
	Learned renderer	0.0148	0.111	0.877	18.3	11.4
	Self-training	0.0175	0.135	0.846	19.7	13.1
	MeshTac (ours)	**0.0075**	**0.078**	0.904	**16.2**	9.9
	no renderer	0.0076	0.081	**0.911**	16.3	**9.2**
50	Real only	0.0088	0.083	0.899	16.9	**9.0**
	MeshTac (ours), no renderer	**0.0041**	**0.061**	**0.933**	**14.8**	9.7
100	Real only	0.0051	0.066	0.932	15.1	**8.4**
	MeshTac (ours), no renderer	**0.0034**	**0.056**	**0.944**	**14.4**	9.1

E. A Second Sensor

MeshTac. On GelSlim 4.0 ($1{,}620$ test frames) we retrain MeshTac with the same aligner settings (Table VII, Fig. 4d). The public depth labels of this dataset are not mesh projections, so we re-registered them: the mesh pose that best explains each label was fitted (0.018 mm mean residual) and canonicalized against the object's rotational symmetry, and every method was retrained and tested on these mesh-registered labels. The few-shot teacher is already accurate on this sensor (label IoU 0.78, 0.85, 0.90, 0.93 on the unlabeled frames at $K = 10$, 25, 50, 100; 0.55 at $K = 25$ on sensor A), yet the aligner improves on it at every budget (IoU 0.85, 0.91, 0.94, 0.95; pose error $2.3°$, $1.4°$, $0.9°$, $0.8°$ against 6.6–$11.4°$). At $K = 25$ the student lowers depth MSE from 0.0179 ± 0.0004 to 0.0075 ± 0.0001 and contact MAE from 0.127 ± 0.001 to 0.078 ± 0.0003 mm over real-only training (three seeds) and raises IoU from 0.849 to 0.903. The baselines gain far less: the learned renderer trained on the 25 pairs reaches 0.111 mm (IoU 0.877) and self-training on the teacher's depth does not improve on real data (0.135 mm, 0.846). Without any synthetic frame, MeshTac lowers contact MAE from 0.180, 0.127, 0.083, 0.066 to 0.121, 0.081, 0.061, 0.056 mm at $K = 10$–100 (-15 to -36%) and depth MSE by 33–58%. At $K = 25$ and 50, its

TABLE VIII: **Sensor B (GelSlim 4.0).** 1,620 test frames of the same 12 objects. Top: sensor-A models applied zero-shot. Below: models trained on K sensor-B real frames per object, alone, with the sensor-A physical renderer, with a learned renderer trained on the same K sensor-B pairs, or with self-training on teacher labels (MeshTac without its alignment step, Sec. IV; labeler: the GAN-25 model; 50 epochs). Rotation after removing a per-object offset.

K	Method	Depth MSE↓	Contact MAE↓	IoU ↑	Normal (deg)↓	Rot. (deg)↓
0	Sensor-A model, real only	0.1486	0.513	0.280	25.7	11.6
	Sensor-A model, Blender only	0.1175	0.413	0.370	34.3	11.6
10	Real only	0.0248	0.165	0.803	20.5	11.1
	Blender (sensor-A calib.)	0.0266	0.168	0.803	20.0	11.5
	GAN-10 (sensor B)	0.0237	0.155	0.809	19.0	11.3
25	Real only	0.0152	0.139	0.824	17.2	10.9
	Blender (sensor-A calib.)	0.0147	0.125	0.846	16.5	11.2
	Blender (domain rand.)	0.0146	0.126	0.830	16.3	10.9
	GAN-25 (sensor B)	0.0107	0.100	0.846	14.6	10.4
	Teacher-label self-training, model only	0.0098	0.100	0.841	14.8	11.6
	Teacher-label self-training	0.0102	0.098	0.840	14.6	11.8
50	Real only	0.0063	0.081	0.860	13.2	9.9
	Blender (sensor-A calib.)	0.0058	0.077	0.858	12.8	9.7
	GAN-50 (sensor B)	0.0045	0.068	0.866	12.1	10.1
100	Real only	**0.0032**	0.058	0.861	**11.0**	**7.5**
	Blender (sensor-A calib.)	0.0033	**0.057**	0.868	11.0	9.3
	Blender (domain rand.)	0.0036	0.062	0.866	11.4	10.6
	GAN-100 (sensor B)	0.0034	0.060	**0.870**	11.2	10.1

contact MAE is no higher than real-only training with twice as many labels. Rotation error improves at 10 and 25 labels ($14.0^\circ \to 9.5^\circ$, $13.4^\circ \to 9.2^\circ$) but not beyond (9° either way). Scored against the original public labels, MeshTac keeps its advantage (contact MAE $0.137 \to 0.098$ mm at $K = 25$).

Renderer baselines (original labels). Sensor B has a marker array, a different LED layout and a smaller field of view (Table VIII). *Zero-shot*, sensor-A models recover contact only partially (IoU 0.28 for the all-real model, 0.37 for the simulation-only model: the model trained on the most sensor-A real data transfers worst) and rotation not at all (residual $\approx 11.5^\circ$ for every model, the value a constant predictor attains on the $\pm 22.5^\circ$ label range). *With sensor-B real pairs*, the K-shot picture of Sec. V-A reappears: a learned renderer trained on the sensor's own $K = 25$ pairs lowers contact MAE from 0.139 to 0.100 mm (-29%), normal error from 17.2° to 14.6° and raises IoU from 0.82 to 0.85; at $K = 10$ the gain is 6%, at $K = 100$ real data alone is already at 0.058 mm and simulation neither helps nor hurts dense error. The sensor-A physical renderer, calibrated to a different sensor, still helps at $K = 25$ (0.125) but less; the domain-randomized renderer matches it (0.126) without using any real data from either sensor. At $K = 100$ real data alone is already at 0.058 mm and simulation neither helps nor hurts dense error; the domain-randomized renderer (0.062) is close. Rotation is not improved by either renderer and is degraded at $K = 100$ ($7.5^\circ \to 9$–10°), as on sensor A. The transition is earlier on sensor B ($K \approx 25$ vs. 50), plausibly because its uniform 1 mm indentation and larger contact regions make the depth-to-image mapping easier to learn from few pairs.

Self-training (original labels). With the original labels and the GAN-25 model as labeler, teacher-label self-training lowers depth MSE from 0.0152 (real only) to 0.0098 and contact MAE from 0.140 to 0.100 mm at $K = 25$ without synthetic data, i.e. it reaches its labeler (0.0107, 0.100) but does not pass it in contact error; with the mesh-registered labels and a real-only teacher it does not improve on real data at all (Table VII). In both protocols the student inherits the localization of its labeler, which is what alignment corrects.

F. Qualitative Results

Fig. 6 shows predictions at $K = 25$ on held-out frames of both sensors; for every object the displayed frame is the one with the object's median improvement in contact MAE, This fixes frame selection within each of the three displayed objects per sensor; it does not summarize the full object distribution. Real-only models blur or break thin ridges and misplace junctions; the MeshTac student recovers their extent and position. On sensor B the real-only model is already close and the gain shows at junctions and at the ends of ridges.

G. Baseline Study: Unseen Objects

Three objects are removed from all real training while their meshes remain available to simulation (Table IX).

Pose. Without any data of the three objects the pose head is at chance (64.7°). Simulated contacts of the three meshes bring it to 6.8° with the physical renderer—with no-contact illumination calibration and manually tuned contact parameters—and to 5.7° with the strict learned renderer; the leaky renderer (4.6°) buys little, and the geometry control (same renderer, meshes removed) stays at 50.6°. The geometry control supports mesh coverage as a major source of the pose improvement. Replacing the calibrated renderer with domain-randomized sensor parameters (random LED colors and strengths, gel optics randomized per session, geometry pinned) gives 8.9°, between the calibrated (6.8°) and single-rotation (10.3°) rows; calibration helps by providing a consistent appearance from which the encoder extracts geometry.

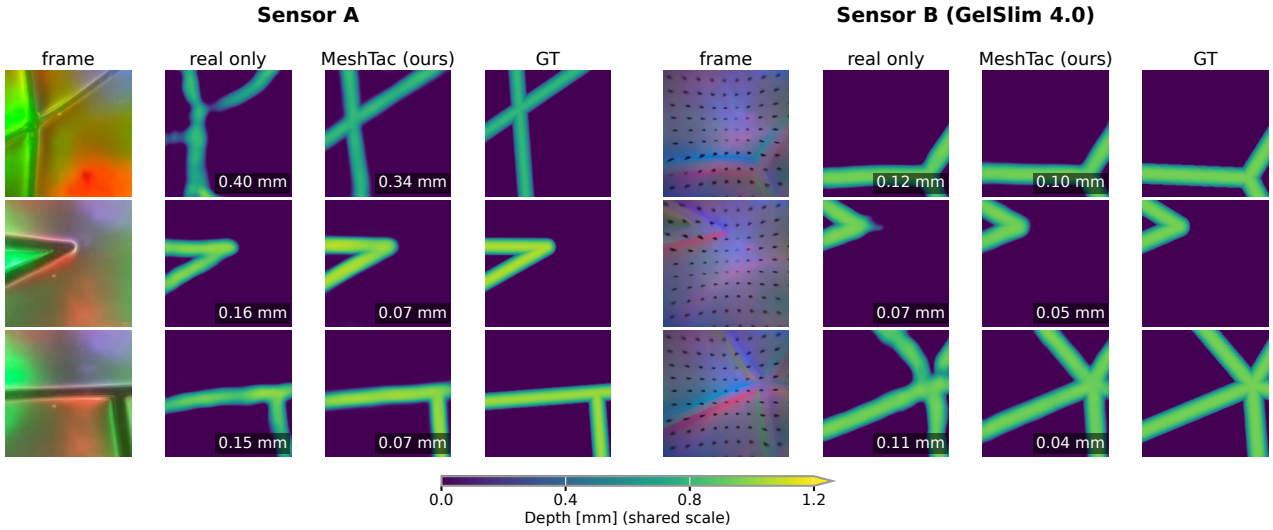


Fig. 6: **Depth predictions.** $K = 25$, held-out frames of three selected objects per sensor. Each example has the median contact-MAE improvement for its object. Numbers are contact MAE (mm); depth maps share a color scale. Both displayed MeshTac models use synthetic data; the sensor-B curves in Fig. 4d use the renderer-free variant.

Rotation coverage. How much of the pose space must simulation cover? Restricting simulation to one real rotation per object leaves $10.3°$, two rotations $7.5°$, all twelve $6.8°$ (Blender rows of Table IX); rendering the learned renderer's inputs after rotating the depth map (rotate-then-render) gives the best leakage-free pose, $4.4°$. Coverage of the rotation range, not the number of synthetic frames, is the operative variable.

Depth and normals. Contact MAE on the unseen objects is 0.36 mm for real-only training, three times the seen-object value; the plain renderers reduce it by at most 7% and the geometry control by almost as much (0.340), suggesting that these small dense gains may reflect regularization as well as object coverage. The domain-randomized Blender renderer improves contact MAE to 0.342 and IoU to 0.601, matching the calibrated renderer (0.346, 0.592), suggesting that appearance diversity from randomized sensor parameters acts as regularization even without calibration data. A multi-sensor GAN trained on 100 simulated sensor designs does not improve over single-sensor baselines (0.358, 0.579); mixing simulated sensors cannot replace real pairs. The one leakage-free renderer that helps significantly is the background-conditioned GAN-LOO: depth MSE -13%, contact MAE 0.323, IoU 0.633, for one background frame per session; it remains far from the leaky reference (-30%), which has seen real contacts of the objects. Normal error follows contact MAE. Peak-depth error is 0.05–0.08 mm for every row: the models estimate *how deep* correctly and err in *where* the contact lies.

Object classification. We add a 12-class classification head (LayerNorm + MLP on the class and mean patch tokens) and train with cross-entropy. With real data of the nine objects, the model classifies them at 94% validation accuracy, but the three held-out objects score 0%—even when simulation provides their contact patterns through either the calibrated or learned renderer. Pose estimation transfers to unseen objects through geometry, but classification requires the appearance of the real contact, which no leakage-free renderer reproduces.

H. Baseline Study: Real-Data Budget of a Renderer

Tables I and II, and Fig. 4 vary the real budget of the renderers. Three regimes appear.

Zero-shot. Simulation alone transfers poorly for dense geometry (contact MAE 0.41 mm vs. 0.11 with the full real set); the leaky renderer halves this but has seen $4{,}197$ real pairs.

Few-shot ($K \leq 25$). Neither leakage-free renderer improves dense error. At $K = 25$ GAN-25 lowers depth MSE by 3% (0.0536 ± 0.0013 vs. 0.0551 ± 0.0010 over seeds) but not contact MAE or IoU; the physical renderer does the same. With $K \leq 25$ the real training loss reaches 10^{-4} within ten epochs while validation error is flat from epoch five: the model memorizes the few real frames, and synthetic frames with a different contact appearance do not regularize that memorization. Simulation does help pose at $K = 10$ ($9.6° \to 5.2$–$5.9°$), as in Sec. V-G.

Moderate budget ($K \geq 50$). The learned renderer now helps: at $K = 50$ depth MSE $0.0482 \to 0.0434$ (-10%), contact MAE $0.283 \to 0.260$, IoU $0.69 \to 0.72$; at $K = 100$ depth MSE $0.0369 \to 0.0311$ (-16%; $0.0366 \to 0.0299$ over seeds, -18%), contact MAE $0.235 \to 0.210$, IoU $0.74 \to 0.77$, and normal error improves in step. The physical renderer gives -1 to -5% MSE. The learned renderer thus needs ≈ 50 labeled contacts per object before its data is worth more than the frames it was trained on, and at every $K \geq 50$ simulation *worsens* rotation by 0.2–$1.3°$.

I. Leakage, a Fidelity Proxy, and Negative Results

Cost of leakage. Across Tables IX and I the leaky renderer reports depth MSE reductions of 24–26% at the reported K-shot budgets and 30% on unseen objects; the strict numbers

TABLE IX: **Leave-object-out.** Errors on 1,244 real frames of three objects with no real training data. Rows add simulated contacts of all 12 meshes to the real frames of the nine other objects, unless noted. The multi-sensor row is a 50-epoch run. Normal error is inside the contact region. Bold: best leakage-free.

Method	Depth MSE↓	Contact MAE↓	Contact IoU↑	Normal (deg)↓	Rot. (deg)↓	Trans. L_1↓
Real only (9 objects)	0.0665	0.356	0.582	30.4	64.7	0.099
Blender	0.0657	0.346	0.592	30.3	6.8	**0.032**
Blender (domain rand.)	0.0635	0.342	0.601	30.4	8.9	0.038
Blender, bg-norm.	0.0672	0.348	0.589	30.3	5.3	0.033
Blender, 1 rotation / object	0.0645	0.354	0.585	30.4	10.3	0.039
Blender, 2 rotations / object	0.0670	0.345	0.590	30.8	7.5	0.042
GAN-LOO	0.0631	0.331	0.618	29.9	5.7	0.035
w/o held-out meshes (control)	0.0647	0.340	0.604	30.0	50.6	0.111
Bg-cond. GAN-LOO	**0.0581**	**0.323**	**0.633**	**29.1**	4.8	0.038
GAN-LOO, rotate-then-render	0.0625	0.345	0.606	29.6	**4.4**	0.034
Multi-sensor GAN-LOO	0.0695	0.358	0.579	30.9	8.6	0.039
Blender ⊕ GAN-LOO	0.0661	0.349	0.601	30.7	5.6	0.034
Refiner (0 labels)	0.0673	0.349	0.590	30.5	7.5	0.046
Refiner on domain rand. (0 labels, no calib.)	0.0691	0.353	0.583	30.8	7.2	0.045
GAN-full (leaky)	*0.0467*	*0.277*	*0.676*	*25.8*	*4.6*	*0.035*

are 0–3% for $K \leq 25$, 10–16% at $K \geq 50$ and 5–13% on unseen objects. The bias is largest where the pipeline is most attractive—few or no real contacts of the target—because GAN-full receives additional labeled training contacts. In LOO it also receives contacts from the held-out objects. The lower rendering error does not establish memorization of validation frames, which are excluded from its training set.

A fidelity proxy. Rendering the depth maps of the real validation pairs and comparing with the real images gives an image error per renderer. Fig. 7 plots the contact-region L1 against the downstream depth MSE for every learned-renderer variant: lower image error is associated with lower downstream MSE in this collection. Over the strict paired renderers, pixel metrics predict downstream error best (Spearman $|\rho|$: contact L1 0.95 over the 14 renderers of Fig. 7, 0.97 over the nine original ones; PSNR 0.98, full-image L1 0.95), perceptual metrics worse (SSIM 0.87, LPIPS

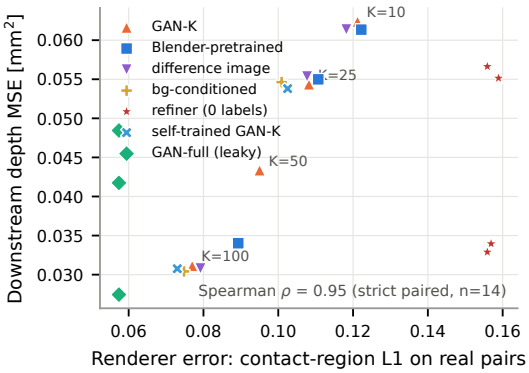


Fig. 7: **Renderer fidelity predicts downstream error.** Contact-region L1 between rendered and real images on held-out real (depth, image) pairs versus depth MSE of the downstream model trained with that renderer's data, for every learned-renderer variant and budget.

0.87, contact-cropped 0.90); adding the three leaky renderers, whose fidelity is identical while their downstream error varies with K, lowers every correlation to 0.75–0.82, These pooled correlations mix budget and renderer changes and do not isolate the causal effect of image fidelity. The unpaired refiner is the one exception: it is scored against the gel-warped physics render rather than its own input, so its contact L1 (0.16) is not comparable (Sec. V-B). The image metric was used to screen the variants below before downstream training.

Which factor limits few-shot fidelity?

Table X tests the hypotheses. *(i) Missing physical prior.* Pretraining the GAN on 30k Blender pairs, conditioning it on the Blender image of the same contact, predicting a residual on top of that image, or conditioning on Blender image and background all leave contact-region fidelity unchanged or worse at $K \in \{10, 25\}$ (the physical renderer's random gel-warp of ± 3 pixels means the conditioning image is not pixel-aligned with the target, which may limit this family). *(ii) Session illumination drift.* Session backgrounds differ by 0.05–0.10 L1, more than the best renderer's total error (0.03); normalizing frames to a canonical background or predicting difference images improves full-image L1 by 13–21% but leaves contact-region L1 and contact MAE unchanged (the frozen encoder is robust to the illumination field). Conditioning the renderer on the background does help on unseen objects (Sec. V-G) because it lets a renderer that never saw the object reproduce its session's illumination. *(iii) Training-side changes.* A contact-mask head with a gradient-matching loss, Blender pretraining followed by real fine-tuning, and fine-tuning with an L2-SP anchor change $K = 25$ contact MAE by at most ± 0.02 mm. *(iv) Domain randomization.* Randomizing the sensor's LED colors, strengths and gel optics per session while keeping the geometry stage fixed (Table X, "domain rand.") matches the calibrated renderer's contact MAE at $K = 25$ (0.320 vs. 0.329) and on unseen objects (0.342 vs. 0.346). On sensor B at $K = 25$ the match is even closer (0.126 vs. 0.125). The calibrated renderer's advantage on

TABLE X: **Appearance-side and training-side variants** at $K = 25$ (real validation split) and leave-object-out (unseen objects). Fidelity is the contact-region L1 of Fig. 7; physics-guided variants were stopped at the fidelity stage. Only the background-conditioned renderer improves unseen-object depth.

	$K = 25$		Unseen objects		
Variant	Fidelity ↓	Contact MAE↓	Fidelity ↓	Contact MAE↓	Rot. (deg)↓
Real only	–	0.312	–	0.356	64.7
GAN-K / GAN-LOO	0.108	0.321	0.068	0.331	5.7
Blender-pretrained	0.111	0.335	0.084	0.356	8.5
difference image	0.108	0.339	0.066	0.346	6.3
background-conditioned	**0.101**	0.330	**0.062**	**0.323**	**4.8**
physics-conditioned	0.114	–	–	–	–
physics residual	0.119	–	–	–	–
physics + background	0.104	–	–	–	–
self-trained (+ pseudo pairs)	0.102	0.326	–	–	–
Refiner, unpaired (0 labels)	0.159	0.297	0.159	0.349	7.5
+ self-regularization	–	–	0.156	0.360	7.4
on domain-rand. render	–	–	0.159	0.353	7.2
Blender (calibrated)	–	0.329	0.179	0.346	6.8
Blender (domain rand.)	–	0.320	–	0.342	8.9
Blender, bg-norm.	–	0.338	–	0.348	5.3
Real only, mask head + grad. loss	–	0.309	–	–	–
GAN-K, mask head + grad. loss	–	0.302	–	–	–
Blender pretrain → real fine-tune	–	**0.294**	–	–	–
+ L2-SP	–	0.307	–	–	–

pose (6.8° vs. 8.9° on unseen objects) is consistent with a less noisy silhouette. Domain randomization is thus a useful baseline without appearance calibration when calibration data is not available, but it cannot close the gap to the learned renderer. *(v) Contact shading.* The appearance *inside* the contact remains a plausible limitation (Fig. 2): strict renderers separate from real-only training only after enough contacts, additional paired labels improve renderer performance, and the tested synthetic-data configurations do not reliably transfer object classification. These observations implicate appearance mismatch, but do not isolate contact shading as the sole cause.

Unpaired refiner (0 labels). The contact-preserving refiner of Sec. III-E translates the physical render using only unlabeled frames. On unseen objects (Table IX) it does not improve on the calibrated render (contact MAE 0.349 vs. 0.346; on the domain-randomized render 0.353), and its self-regularized variant is slightly worse (0.360): without a single labeled pair the adversarial objective is satisfied by any plausible contact shading, including a fainter one, and the ridge structure that geometry transfer needs is not what the discriminator judges. In the K-shot protocol, refining with the same $K = 25$ frames used as unlabeled images improves over the paired-renderer baselines at that budget (contact MAE 0.297, IoU 0.672, normal 27.1°; Table I), a 5% contact-MAE gain where no paired renderer helps, while using all of U (0.309) or $K = 100$ (0.224 vs. real-only 0.235, GAN-100 0.210) adds little. The refiner's contact-region L1 (0.156–0.159, against the ± 3 px-warped physics render whose identity score is 0.179) is not comparable to the paired renderers' and is excluded from Fig. 7.

Mixing the two renderers. Alternating physical and learned frames (Blender $\oplus$ GAN) yields the average of the two on dense error and the better of the two on pose (5.6° on unseen objects, equal to GAN-LOO; 9.8° across sessions, the lowest of all renderers except background normalization), This suggests a task-dependent effect of mixing, rather than a consistent gain across outputs.

VI. Limitations and Conclusion

MeshTac requires the identity and mesh of each object during pseudo-labeling and optional test-time refinement. Our evaluation covers 12 ridge-pattern objects and does not establish performance on curved or textured contacts, unknown objects, or a manipulation task. GT is mesh-derived on both sensors; sensor-B targets are re-registered public labels. Validation labels are also needed for checkpoint selection. Alignment can fail, particularly across sessions, and repeated ridge patterns leave some translations ambiguous.

Within this scope, learning to align a mesh to a real tactile image provides useful supervision from few registered contacts. With 10–25 training labels per object, MeshTac reduces contact MAE by 13–20% and 33–36% on the two sensors. Ablations attribute the improvement to more accurate geometric labels. Synthetic images provide an additional benefit across sessions, while the aligner can also refine test-time pose. Extending this approach to less constrained contact geometry and evaluating its effect on manipulation are the next steps.

References

[1] W. Yuan, S. Dong, and E. H. Adelson, "Gelsight: High-resolution robot tactile sensors for estimating geometry and force," *Sensors*, vol. 17, no. 12, 2017.

[2] E. Donlon, S. Dong, M. Liu, J. Li, E. Adelson, and A. Rodriguez, "Gelslim: A high-resolution, compact, robust, and calibrated tactile-sensing finger," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2018.

[3] M. Lambeta, P.-W. Chou, S. Tian, B. Yang, B. Maloon, V. R. Most, D. Stroud, R. Santos, A. Byagowi, G. Kammerer, D. Jayaraman, and R. Calandra, "Digit: A novel design for a low-cost compact high-resolution tactile sensor with application to in-hand manipulation," *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 3838–3845, 2020.

[4] S. Wang, M. Lambeta, P.-W. Chou, and R. Calandra, "Tacto: A fast, flexible, and open-source simulator for high-resolution vision-based tactile sensors," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 3930–3937, 2022.

[5] Z. Si and W. Yuan, "Taxim: An example-based simulation model for gelsight tactile sensors," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2361–2368, 2022.

[6] Z. Chen, S. Zhang, S. Luo, F. Sun, and B. Fang, "Tacchi: A pluggable and low computational cost elastomer deformation simulator for optical tactile sensors," *IEEE Robotics and Automation Letters*, vol. 8, no. 3, pp. 1239–1246, 2023.

[7] A. Agarwal, T. Man, and W. Yuan, "Simulation of vision-based tactile sensors using physics based rendering," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 1–7.

[8] J. Xu, S. Kim, T. Chen, A. R. Garcia, P. Agrawal, W. Matusik, and S. Sueda, "Efficient tactile simulation with differentiability for robotic manipulation," in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1488–1498. [Online]. Available: https://proceedings.mlr.press/v205/xu23b.html

[9] I. Akinola, J. Xu, J. Carius, D. Fox, and Y. Narang, "Tacsl: A library for visuotactile sensor simulation and learning," *IEEE Transactions on Robotics*, 2025.

[10] A. Church, J. Lloyd, R. Hadsell, and N. F. Lepora, "Tactile sim-to-real policy transfer via real-to-sim image translation," in *Proc. Conf. Robot Learning (CoRL)*, 2022, pp. 1645–1654.

[11] W. Chen, Y. Xu, Z. Chen, P. Zeng, R. Dang, R. Chen, and J. Xu, "Bidirectional sim-to-real transfer for gelsight tactile sensors with cyclegan," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6187–6194, 2022.

[12] S. Zhong, A. Albini, P. Maiolino, and I. Posner, "Tactgen: Tactile sensory data generation via zero-shot sim-to-real transfer," *IEEE Transactions on Robotics*, vol. 41, pp. 1316–1328, 2025.

[13] A. Shrivastava, T. Pfister, O. Tuzel, J. Susskind, W. Wang, and R. Webb, "Learning from simulated and unsupervised images through adversarial training," in *CVPR*, 2017.

[14] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *ICCV*, 2017.

[15] T. Park, A. A. Efros, R. Zhang, and J.-Y. Zhu, "Contrastive learning for unpaired image-to-image translation," in *ECCV*, 2020.

[16] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017.

[17] H. Gupta, Y. Mo, S. Jin, and W. Yuan, "Sensor-invariant tactile representation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[18] J. Zhao, Y. Ma, L. Wang, and E. Adelson, "Transferable tactile transformers for representation learning across diverse sensors and tasks," in *Proc. Conf. Robot Learning (CoRL)*, 2024.

[19] C. Higuera, A. Sharma, C. K. Bodduluri, T. Fan, P. Lancaster, M. Kalakrishnan, M. Kaess, B. Boots, M. Lambeta, T. Wu, and M. Mukadam, "Sparsh: Self-supervised touch representations for vision-based tactile sensing," in *Proc. Conf. Robot Learning (CoRL)*, 2024.

[20] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *ICML Workshop on Challenges in Representation Learning*, 2013.

[21] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-training with noisy student improves imagenet classification," in *CVPR*, 2020.

[22] Y. Li, G. Wang, X. Ji, Y. Xiang, and D. Fox, "DeepIM: Deep iterative matching for 6D pose estimation," in *ECCV*, 2018.

[23] G. Wang, F. Manhardt, J. Shao, X. Ji, N. Navab, and F. Tombari, "Self6D: Self-supervised monocular 6D object pose estimation," in *ECCV*, 2020.

[24] A. Sipos, W. van den Bogert, and N. Fazeli, "GelSlim 4.0: Focusing on touch and reproducibility," *arXiv preprint arXiv:2409.19770*, 2024.

[25] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134.

[26] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 12 179–12 188.

[27] A. Kendall, Y. Gal, and R. Cipolla, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7482–7491.